Algorithm Design and Analysis (ADA)

CSE222

LECTURE 3 (13/01/2025)

Divide and Conquer Algorithms

1. Divide the problem into smaller subproblems
2. Conquer the subproblems via recursive calls
3. Combine the subproblem answers

A More Intuitive Viewpoint

1. Break the problem into smaller subproblems

A More Intuitive Viewpoint

1. Break the problem into smaller subproblems

2. Assume that some ‘oracle’ solves the smaller subproblems and gives you the solution

A More Intuitive Viewpoint

1. Break the problem into smaller subproblems

2. Assume that some ‘oracle’ solves the smaller subproblems and gives you the solution

3. Try to solve the original problem using these solutions to subproblem

A More Intuitive Viewpoint

1. Break the problem into smaller subproblems

2. Assume that some 'oracle' solves the smaller subproblems and gives you the solution

3. Try to solve the original problem using these solutions to subproblem

Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

A More Intuitive Viewpoint

Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

But where is the oracle ? There is none !

A More Intuitive Viewpoint

Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

But where is the oracle ? There is none !

Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)

A More Intuitive Viewpoint

Key Thing : Forget about solving the smaller subproblem and just focus on combining the solutions to solve the bigger one !

But where is the oracle ? There is none !

Just keep dividing the problem until you can solve using just simple primitive operations (also called Base Cases)

The recursion paradigm keeps combining solutions to smaller subproblems starting with the base cases

Case Study 1 : Sorting

Sorting

Input : An array A of numbers, suppose 1,2,3, … n (for this lecture) in any arbitrary order

Output : The array A in increasing order

Sorting : Example

Input : 1, 3, 5, 2, 4, 6

Output : 1, 2, 3, 4, 5, 6

There exist (relatively) simple algorithms which solves using $O(n^2)$ basic operations

Sorting : Example

Input : 1, 3, 5, 2, 4, 6

Output : 1, 2, 3, 4, 5, 6

There exists simple algorithm which solves in $O(n^2)$

Today : An $O(n \log n)$ time algorithm(mergesort)

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y (assume for simplicity |A| is even)

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .

Now, suppose we obtain X' (X in sorted order) and Y' (Y in sorted order)

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .

Now, suppose (as if by magic), we obtain X' (X in sorted order) and Y' (Y in sorted order)

Somehow 'merge' X' and Y' to obtain sorted A

Pseudocode :

MergeSort (array A, length n)

If n==1 return A

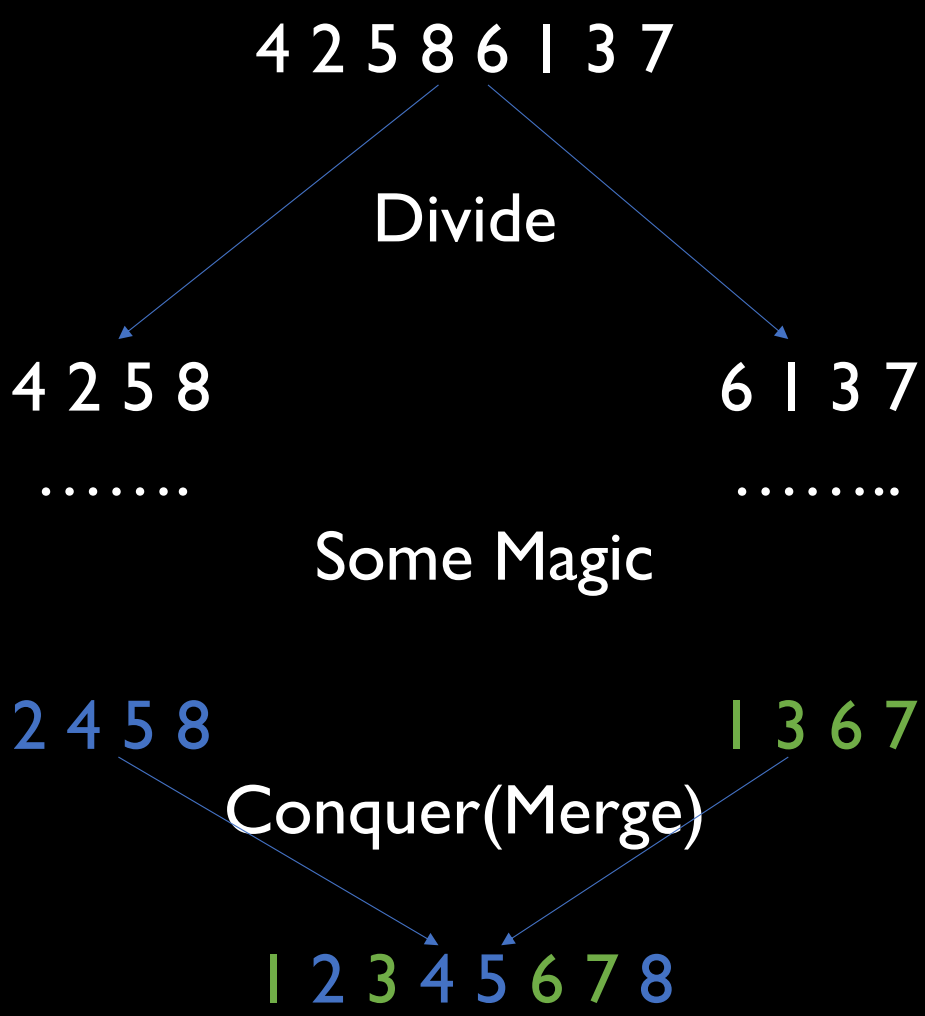
X = A[0, 1,…n/2-1] , Y = A[n/2, …n-1]

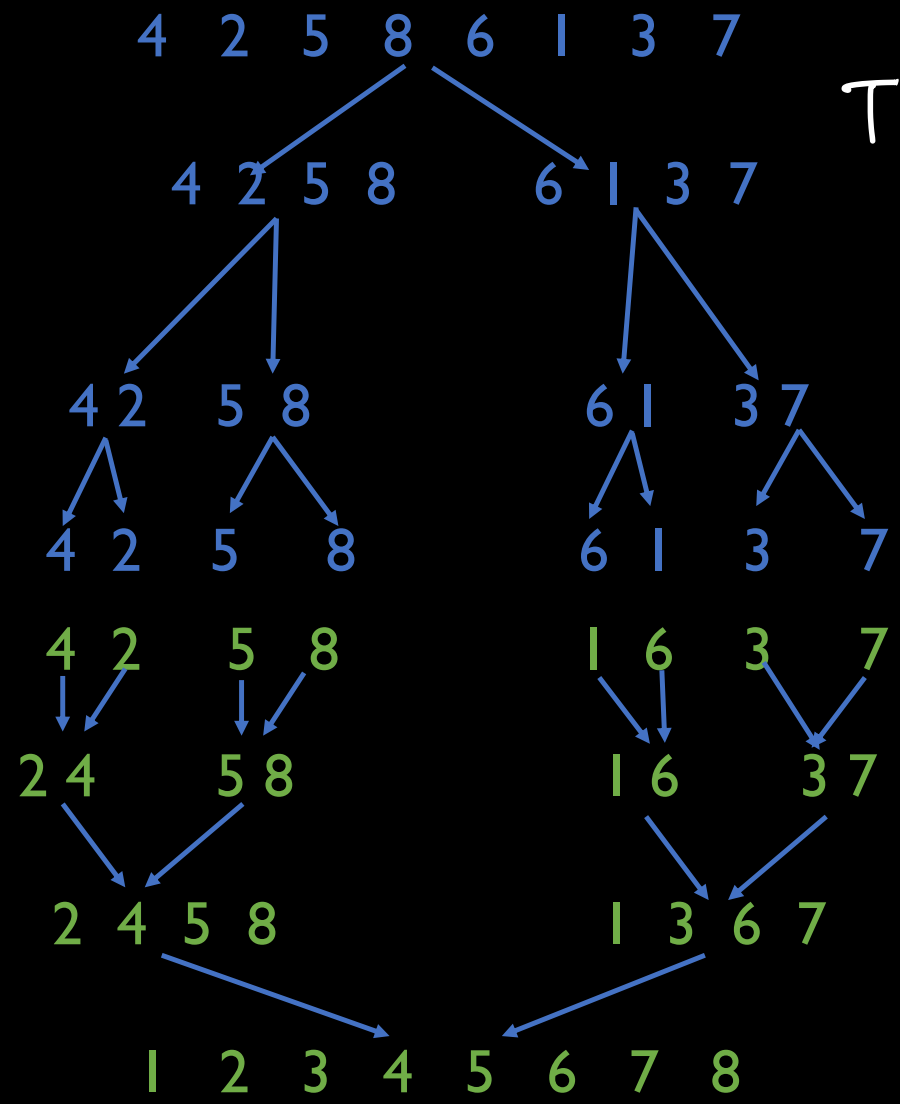
B = MergeSort (X, n/2)

C = MergeSort (Y, n/2)

D = Merge (B, C, n)

Return D


4 2 5 8 6 1 3 7
Divide
4 2 5 8
6 1 3 7
.....
.....
Some Magic
2 4 5 8
1 3 6 7
Conquer(Merge)
1 2 3 4 5 6 7 8



$T(n) = 2 \cdot T(n/2) + ?$

$c \cdot n$

By Master Thm,

$a = 2,\ b = 2,\ d = 1$

$T(n) = O(n \cdot \lg_2 n)$

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .

Now, suppose (as if by magic), we obtain X' (X in sorted order) and Y' (Y in sorted order)

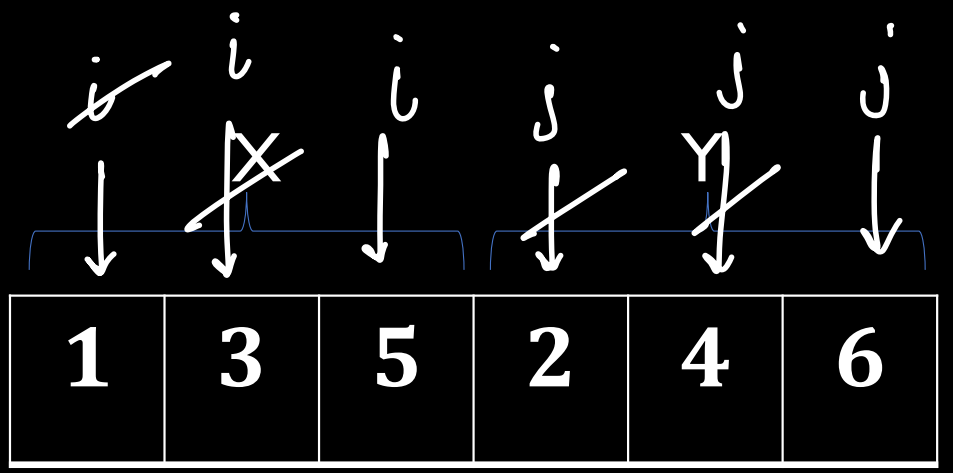
Somehow 'merge' X' and Y' to obtain sorted A

Merge Pseudocode and Example

X , Y : Sorted arrays of length n/2
Z : Output array of length n

```
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        Z(k) = X(i)
        i++
    else
        Z(k) = Y(j)
        j++
```

We need to handle i, j falling off. (EXERCISE)



Z

Runtime Analysis of Mergesort

$T(n) = 2 \cdot T(n/2) +$ runtime of merge.

$\therefore \quad T(1) = 1.$

Runtime Analysis of Mergesort

Modest Goal : How many primitive operations are there in Merge on an array of size n ?

Runtime Analysis of Mergesort

Modest Goal : How many primitive operations are there in Merge on an array of size n ?

X , Y : Sorted arrays of length n/2

Z : Output array of length n

```
i, j = 1                          ---> 2 operations
for k=1 to n                      ---> n operations
    if X(i) < Y(j)                ---> ≤ n operations
        Z(k) = X(i)                    n
        i++                       } ≤ 2n
    else
        Z(k) = Y(j)               }
        j++
```

$4n + 2 \leq 6n.$ $[n \geq 1]$

Runtime Analysis of Mergesort

Modest Goal : How many primitive operations are there in Merge on an array of size p ?

Answer : At most 6p

Runtime Analysis of Mergesort

Modest Goal : How many primitive operations are there in Merge on an array of size p ?

Answer : At most 6p

Claim : The overall number of primitive operations performed by mergesort on an array of size n is at most O(nlogn)

Runtime of Mergesort

Counting Inversions in an Array

Input : An array A of (distinct) natural numbers, suppose 1,2,3, … n (for this lecture) in any arbitrary order

Output : The number of inversions in the array = number of pairs (i, j) such that i < j , but A[i] > A[j]

Counting Inversions : Example

Input : 1, 3, 5, 2, 4, 6

Output : 3

The inversions : (3,2), (5,2), (5,4)

Counting Inversions : Example

Input : 1, 3, 5, 2, 4, 6

Output : 3

The inversions : (3,2), (5,2), (5,4)

Trivial algorithm solves in $O(n^2)$

Counting Inversions : Example

Input : 1, 3, 5, 2, 4, 6

Output : 3

Trivial algorithm solves in $O(n^2)$
Today : An $O(n \log n)$ time algorithm

Counting Inversions : Example

Input : 1, 3, 5, 2, 4, 6

Output : 3

Trivial algorithm solves in $O(n^2)$
Today : An $O(n \log n)$ time algorithm

Q. Can we output all inversions in O(nlogn)?

Counting Inversions : Motivation

E-commerce Websites : Collaborative filtering

Use inversion counting to determine ‘similarity’ between two customers

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .
An inversion pair (i, j) is :

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .
An inversion pair (i, j) is :

1. Left inversion : Both (i,j) in X

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .
An inversion pair (i, j) is :

1. Left inversion : Both (i,j) in X

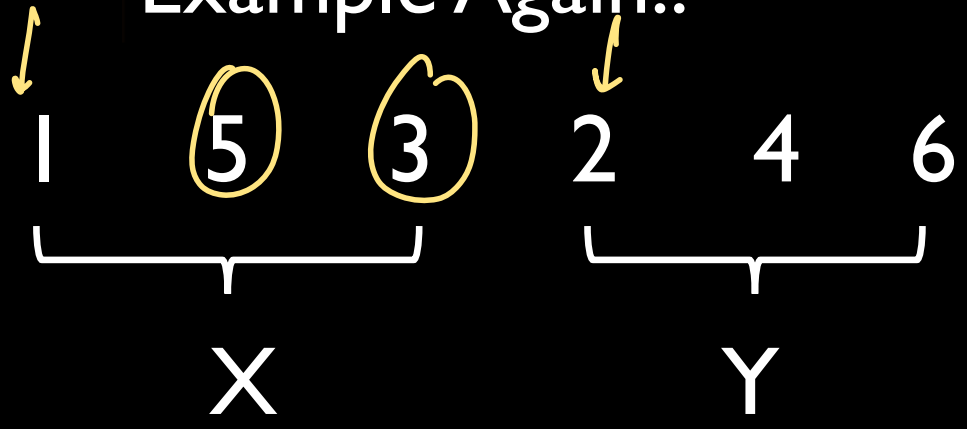
2. Right inversions : Both (i,j) in Y

Key Idea # 1

Suppose A is divided in to X and Y .
An inversion pair (i, j) is :

1. Left inversion : Both (i,j) in X
2. Right inversions : Both (i,j) in Y
3. Split inversions : i in X, j in Y

Example Again..



1. Left inversion : (X(1), X(2))
2. Right inversions : None
3. Split inversions : X(1),Y(0) X(1),Y(1) and X(2),Y(0)

Key Idea for Divide and Conquer

Suppose A is divided in to X and Y .
An inversion pair (i, j) is :

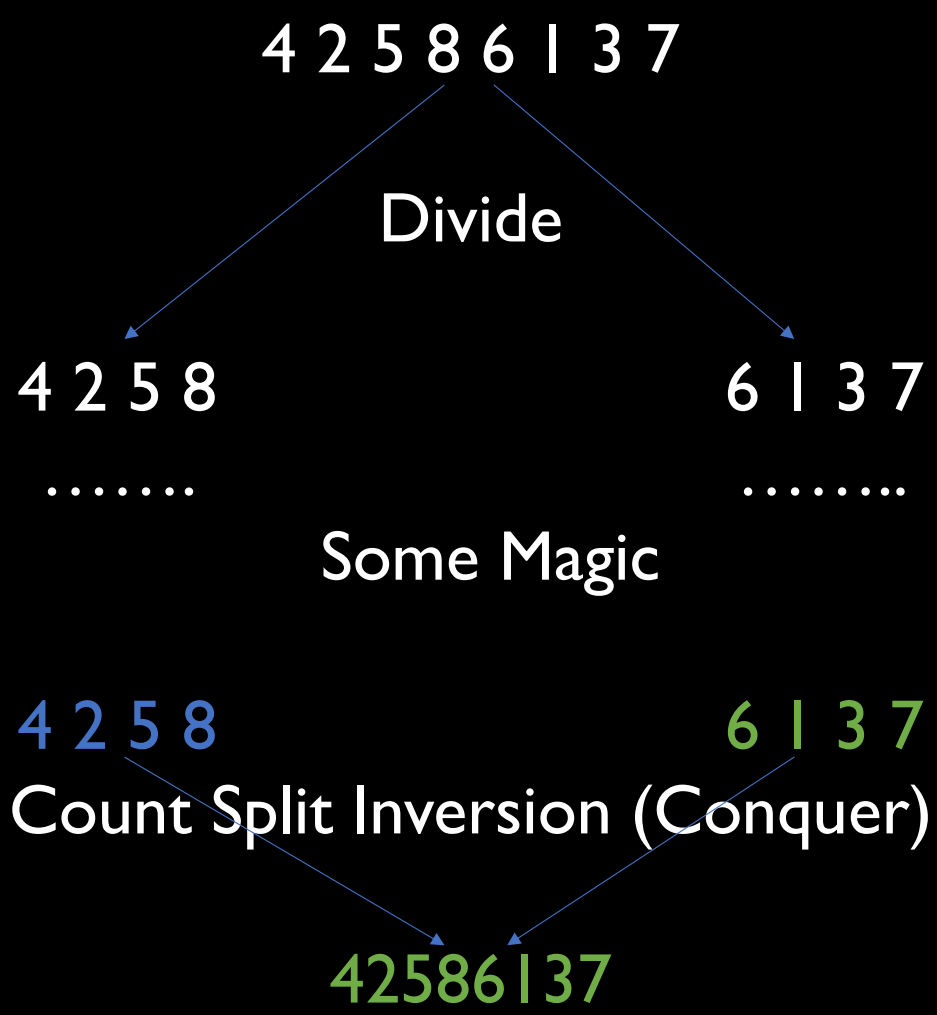
1. Left inversion : Both (i,j) in X
2. Right inversions : Both (i,j) in Y
3. Split inversions : i in X, j in Y

Algorithm : High Level Idea

```
CountInv (array A, length n)
  If n==1  return 0
  X  = A[1,2,…n/2] , Y = A[n/2+1, …n ]

  x = CountInv (X, n/2)
  y = CountInv (Y, n/2)
  z = CountSplitInv (X, Y, n)
  Return x+y+z
```

Goal:
compute
z in c·n
time.


4 2 5 8 6 1 3 7
Divide
4 2 5 8
6 1 3 7
.....
.....
Some Magic
4 2 5 8
6 1 3 7
Count Split Inversion (Conquer)
42586137

4 2 5 8 6 1 3 7

Divide

4 2 5 8 ……. 6 1 3 7 ……..

Some Magic

4 2 5 8 6 1 3 7

Count Split Inversion (Conquer)

42586137

Key Idea # 2

How do we count split inversions in linear time?

Key Idea # 2

How do we count split inversions in linear time?

Piggyback on Merge !

Key Idea # 2

How do we count split inversions in linear time?

Piggyback on Merge !

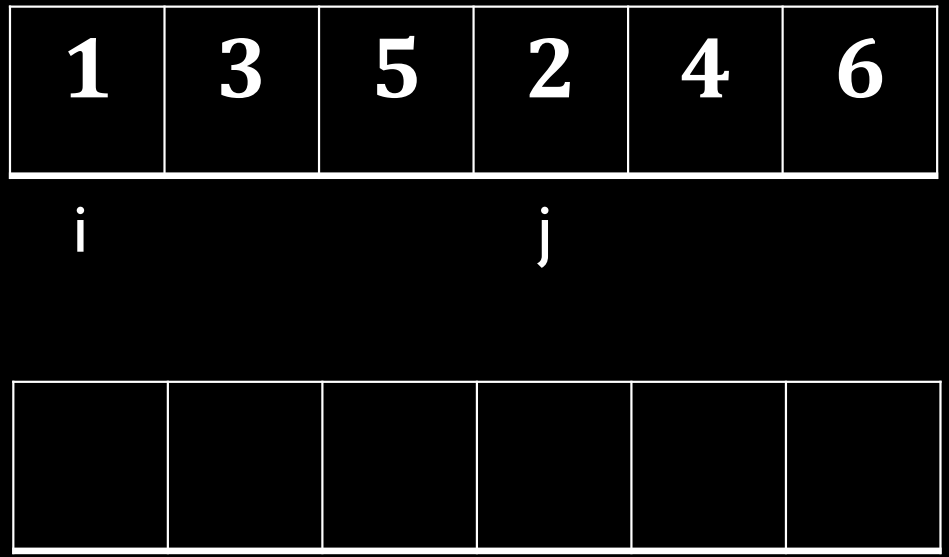
Sort subarrays along with counting inversions

Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :

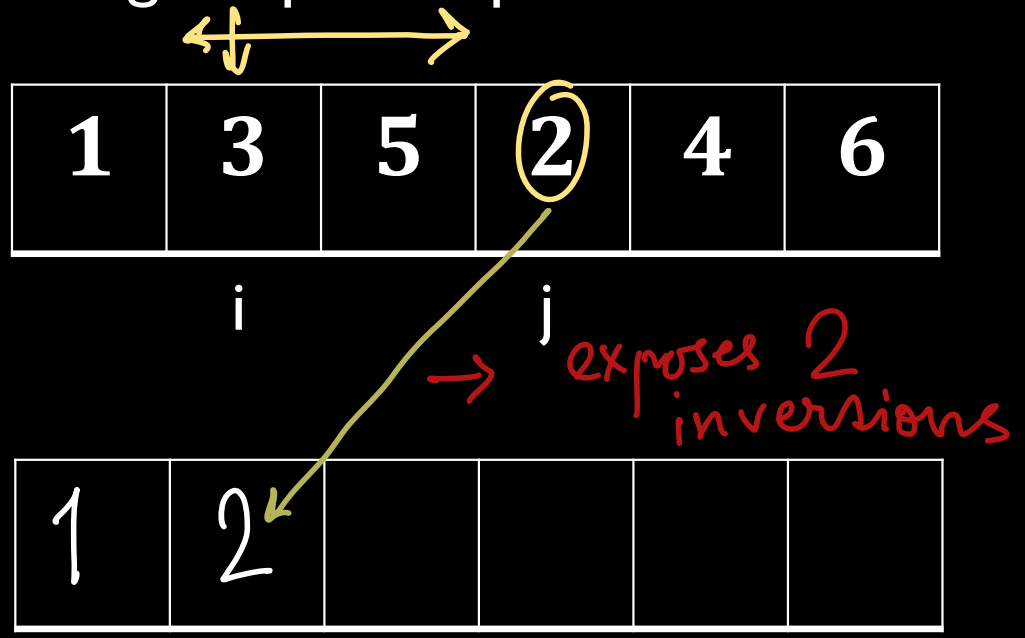


Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :

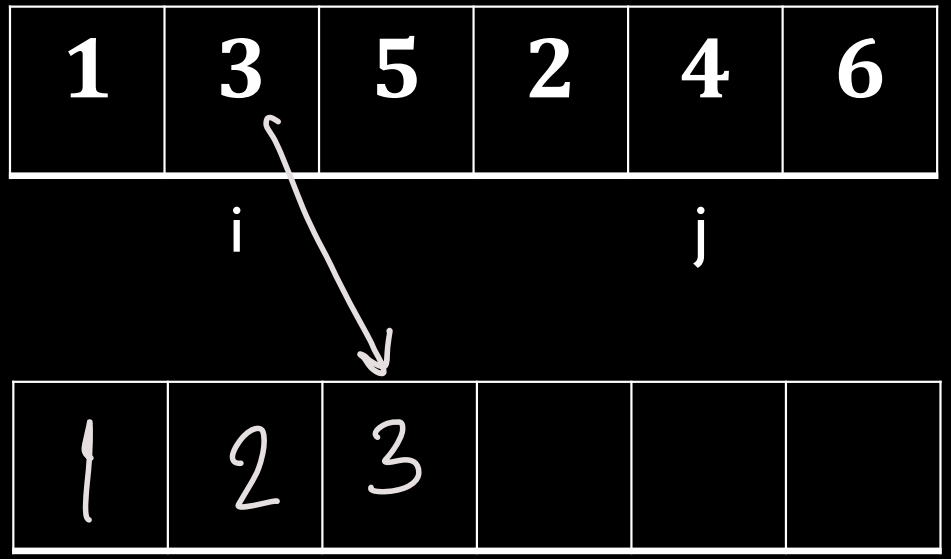


Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :

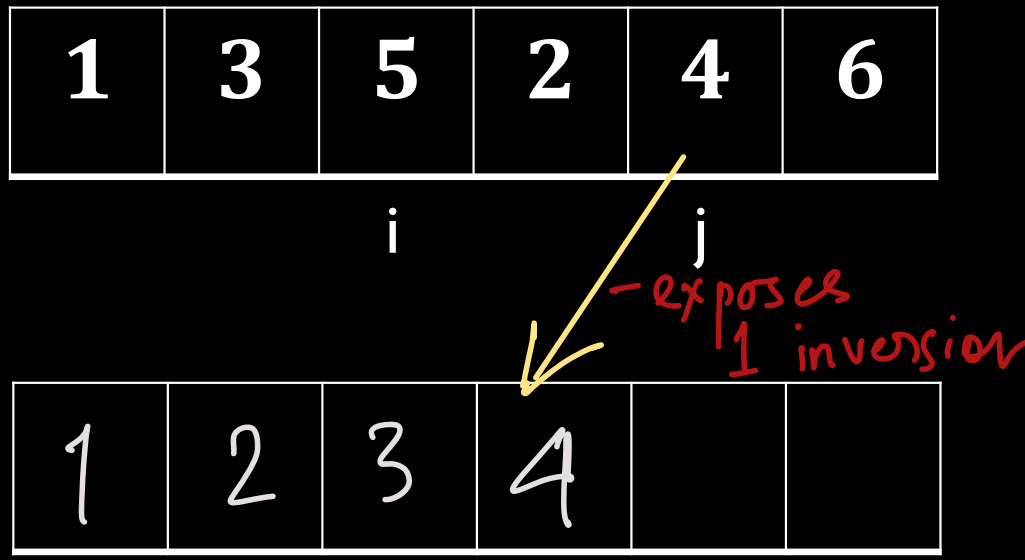


Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :

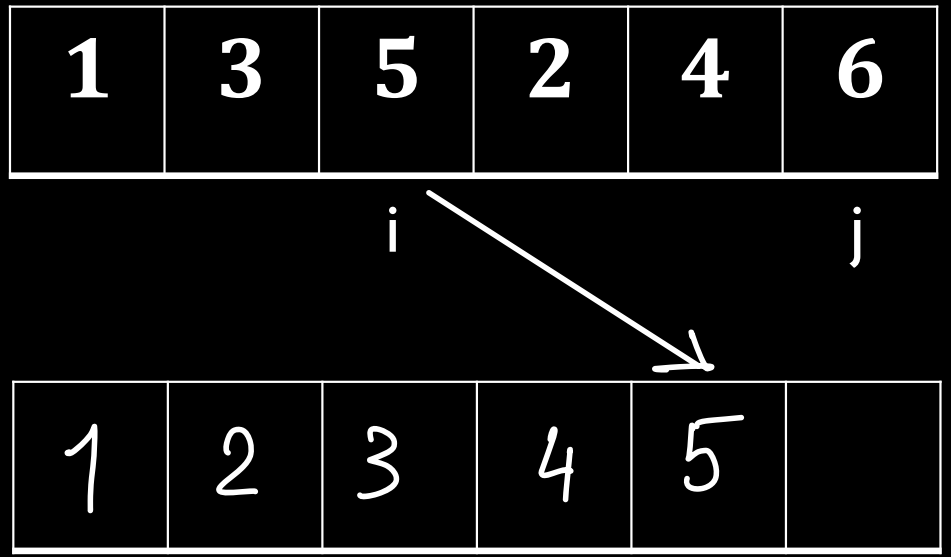


Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :

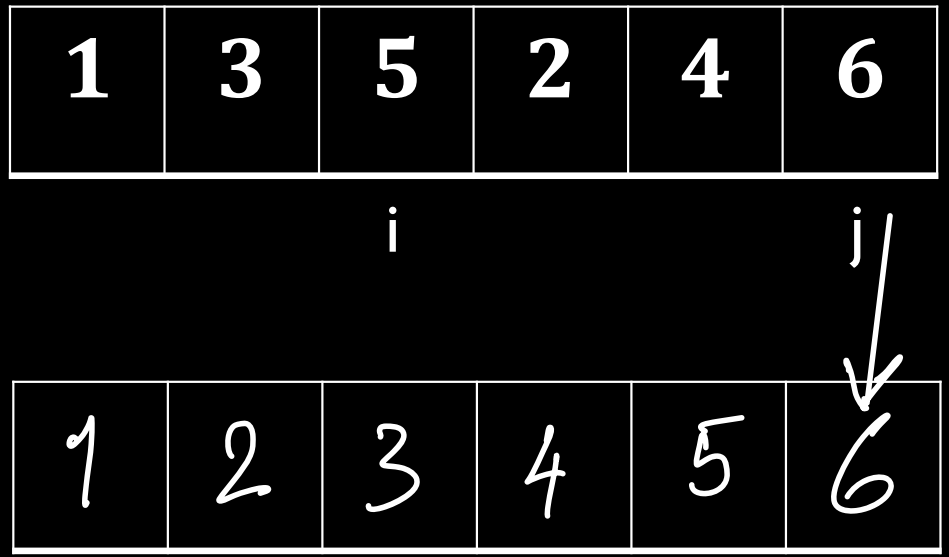


Key Idea # 2 : Why sorted subarrays help ?

Recap of 'Merge' from Mergesort

X , Y : Sorted arrays of length n/2
Z : Output array of length n
i, j = 1
for k=1 to n
    if X(i) < Y(j)
        D(k) = X(i)
        i++
    else
        D(k) = Y(j)
        j++

Merge exposes split inversions :



A General Claim

Claim. The split inversions involving an element y in the subarray Y is precisely n/2-i+1, where i is position of the first pointer when y is copied to D

Proof: There are two parts in the proof :

1. X(k) < y for all k=1,2,…, i-1 since they were copied to output before y in Merge procedure

2. X(k) > y for all k=i, i+1, …, n/2 since X(i) > y and all the other elements in X is bigger than X(i), X being sorted

Modified Merge Procedure

CountAndMerge

X , Y : Sorted arrays of length n/2

Z : Output array of length n, count = 0

i, j = 1

for k=1 to n

  if X(i) < Y(j)

    D(k) = X(i)

    i++

  else

    D(k) = Y(j)

    j++

    count = count + (n/2 – i + 1)

Return count

Algorithm :

$T(n) = 2T(n/2) + cn$

SortAndCountInv (array A, length n)

$T(n) = n + c \cdot n/2 \cdot \log n/2$

If n==1 return 0

X = A[1,2,…n/2] , Y = A[n/2+1, …n]

(B, x) = SortAndCountInv (X, n/2)

(C, y) = SortAndCountInv (Y, n/2)

sort(B), sort(C)

(D, z) = CountAndMerge (X, Y, n)

B C

Return x+y+z

D

$T(n) = 2 \cdot T(n/2) + c \cdot (n/2 \cdot \log(n/2)) + c_1 \cdot n \cdot$

$T(n) = ?$

Matrix Multiplication : Divide and Conquer

Matrix Multiplication

$$\begin{pmatrix} X \end{pmatrix} \begin{pmatrix} Y \end{pmatrix} = \begin{pmatrix} Z \end{pmatrix}$$

Matrix Multiplication

$$\begin{pmatrix} X \end{pmatrix} \begin{pmatrix} Y \end{pmatrix} = \begin{pmatrix} Z \end{pmatrix}$$

Z_{ij} : The product of ith row of X and jth column of Y

Matrix Multiplication

$$\begin{pmatrix} X \end{pmatrix} \begin{pmatrix} Y \end{pmatrix} = \begin{pmatrix} Z \end{pmatrix}$$

Z_{ij} : The product of ith row of X and jth column of Y

$$Z_{ij} = \sum_{k=1}^{n} X_{ik} \cdot X_{kj}$$

Matrix Multiplication

$$\begin{pmatrix} X \end{pmatrix} \begin{pmatrix} Y \end{pmatrix} = \begin{pmatrix} Z \end{pmatrix}$$

What is the best runtime we can hope for ?

O(n-squared) since the input size is this

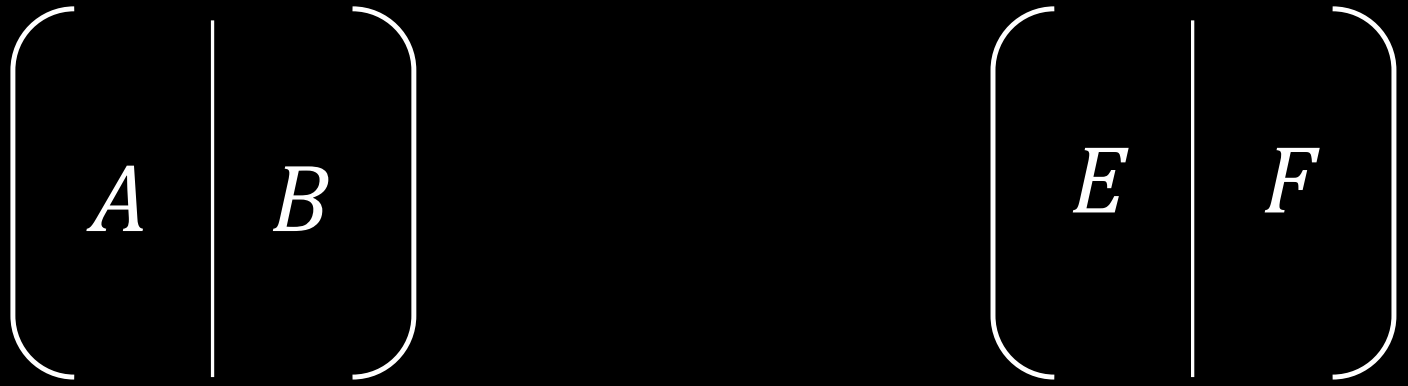
Matrix Multiplication

$$\begin{pmatrix} X \end{pmatrix} \begin{pmatrix} Y \end{pmatrix} = \begin{pmatrix} Z \end{pmatrix}$$

A naïve iterative algorithm :

Just run 3 loops : two for i and j and the third one for k

Divide and Conquer Idea



Divide and Conquer Idea

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \qquad \begin{pmatrix} E & F \\ G & H \end{pmatrix}$$

$$\begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}$$

Recursive algorithm I

$$\begin{pmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{pmatrix}$$

Step I : Recursively Compute 7 cleverly chosen products.

Step II : Perform additions and subtraction with total runtime $O(n^2)$

Recursive Algorithm II : Strassen (1969)

Step I : Recursively Compute 7 cleverly chosen products.

Step II : Perform additions and subtraction with total runtime $O(n^2)$

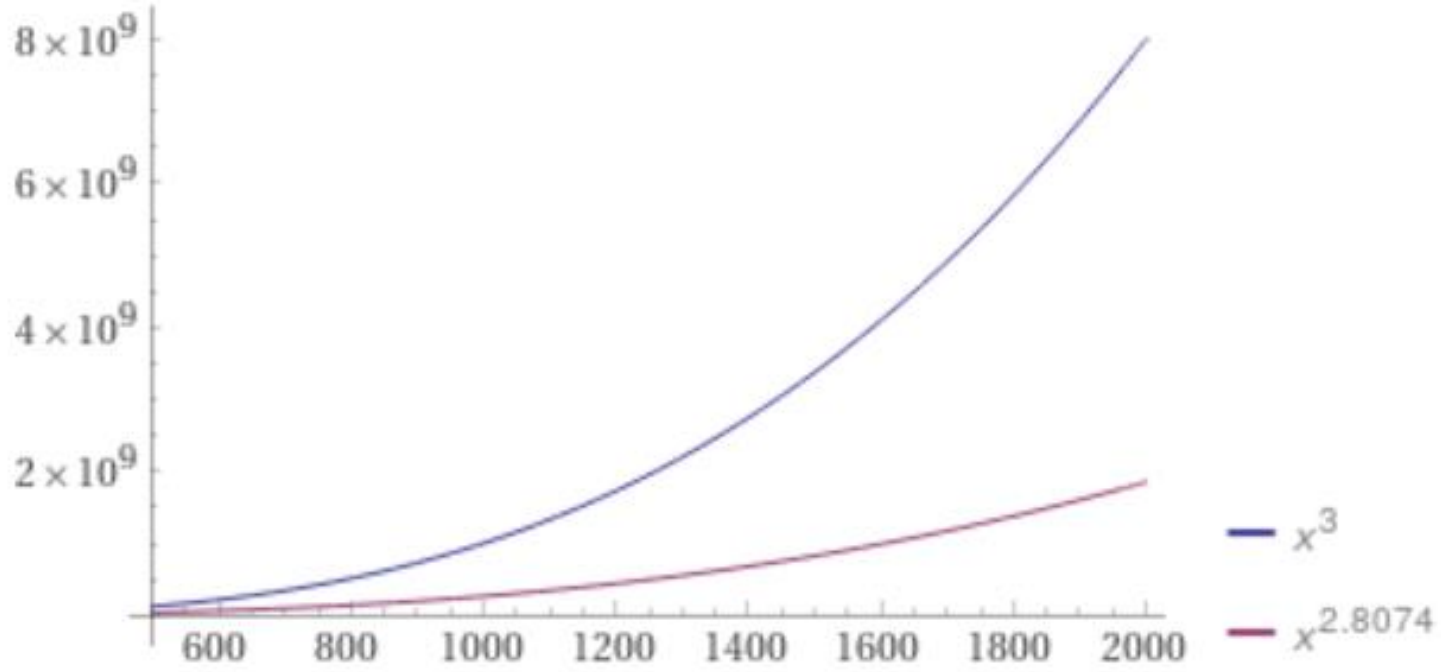
Recursive Algorithm II : Strassen (1969)

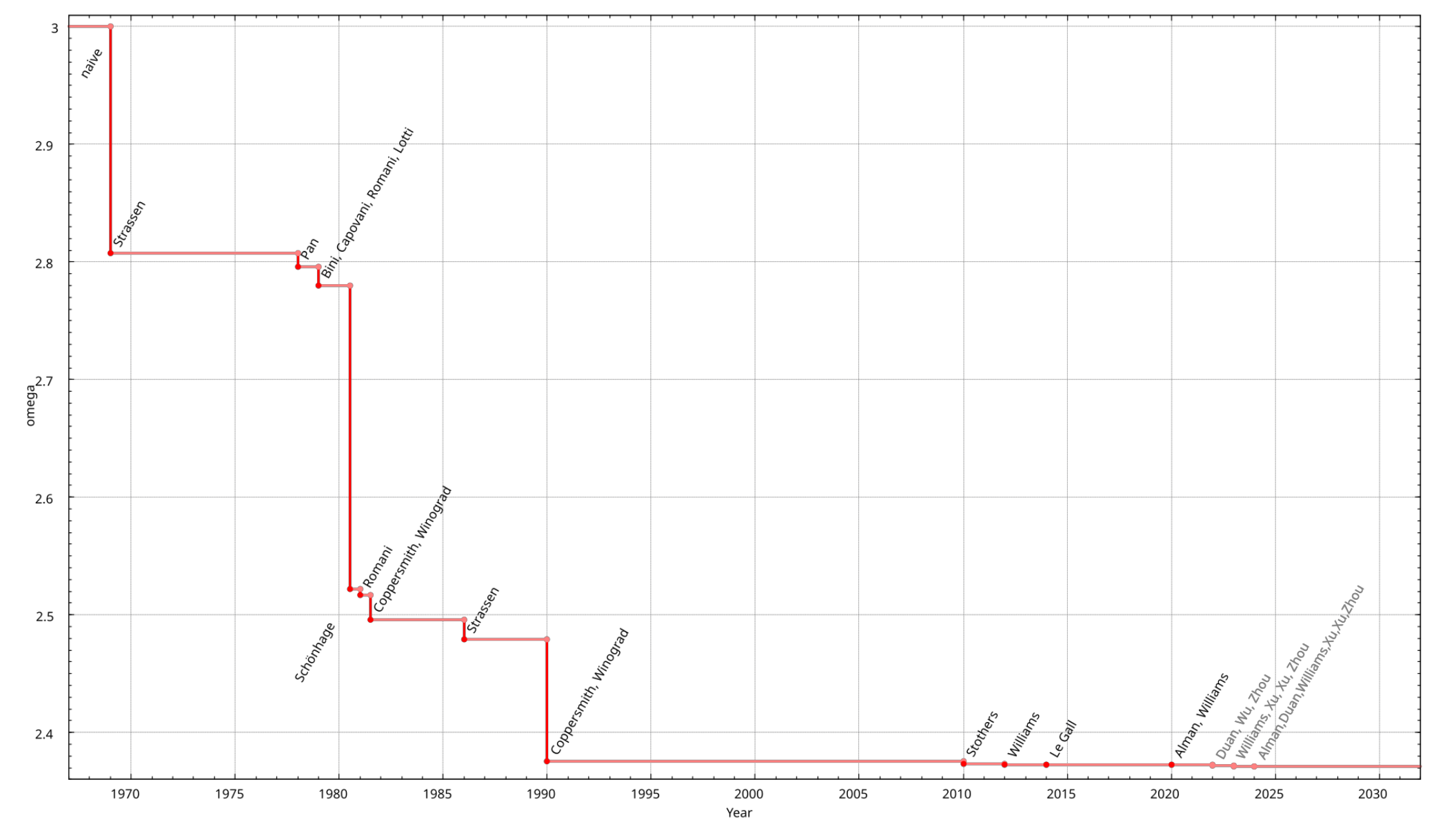
Step I : Recursively Compute 7 cleverly chosen products.

Step II : Perform additions and subtraction with total runtime $O(n^2)$

Question : What is the runtime of this algorithm

Plot:


8 × 10^9
6 × 10^9
4 × 10^9
2 × 10^9
600
800
1000
1200
1400
1600
1800
2000
x^3
x^2.8074


naive
Strassen
Pan
Bini, Capovani, Romani, Lotti
Schönhage
Romani
Coppersmith, Winograd
Strassen
Coppersmith, Winograd
Stothers
Williams
Le Gall
Alman, Williams
Duan, Wu, Zhou
Williams, Xu, Xu, Zhou
Alman,Duan,Williams,Xu,Xu,Zhou
omega
Year
3
2.9
2.8
2.7
2.6
2.5
2.4
1970
1975
1980
1985
1990
1995
2000
2005
2010
2015
2020
2025
2030

Matrix Multiplication : Trivia

Strassen : $O(n^{2.8074})$

At least 15 research papers till December 2020 on this topic !

Current best algorithms

Not practical !

Can we :

1. Get the exponent down to 2 ?
2. Design ‘practical’ algorithms which are faster than Strassen ?